

Introduction to ML and data-driven approaches for Mech and Aero

M. Beneitez & T. Tang

Summer School in Hands-on ML 2026

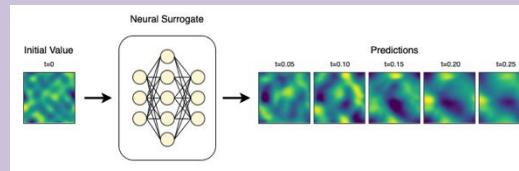
Why?

ML and data-driven methods for engineering and scientific computing



Digital Twins

Live, data-calibrated models that incorporate information from a real physical system



PDE Surrogates

Trained networks that approximate expensive simulations



Learned Control

Controllers that adapt to systems which are too complex or poorly understood

Not a replacement for physics-based modelling, but help with things we are not very good at: **unknown equations, too expensive to solve problems, too non-linear to untangle**

This course

Two weeks, project based to give a hands-on start into something you are interested and hopefully can carry to your own research.

- **This year:** three lecture topics (6h per topic)
 - Time series and symbolic regression
 - Reinforcement learning
 - Introduction to uncertainty quantification
- Lectures in the morning 9-12 (see lecture rooms in the calendar invites)
- Individual project work supported by amazing TAs in the afternoon
- Presentations on Friday 28th of August with the progress. Gift card for the best project!

Machine learning introduction

What is ML?

First-principles simulation

- Derive the governing equations: e.g. conservation of momentum, conservation of mass...
- Discretise the equations and choose a solver: FEM, CFD, network models...
- Obtain a numerical solution and interpret it (often heuristic based)

Data-driven methods

- Map directly from an input to an output. Unknown internal process.
- Doesn't directly consider governing equations, typically not great at generalising
- Performance tied to data quality, quantity and coverage

A view: ML does not replace physics or observation! Complement it whenever equations are unknown, expensive, or intractable. Important to quantify its uncertainty.

Machine learning

What is machine learning?

What is machine learning?

```
100100011101000000101000110111010110
100100111101110000001111100110100100
100001101101111101010011100001101001
111111010000110111001010111100001011
1100111110111111100100001110110110
010000110100110110000110000100010000
01010110011001111011001110100010111
001000010101100101000001000010011110
01110100111110010111010101010111100
100010000101100010101101010111000101
01001000010010101110011100001010000
010110000010011101010010101110110001
01101111101011100010100010100010000
011010011011011010001000101111001101
000101000001100110001100100010010110
10010101010001001110010101010111101
```

ML algorithm

$f(\mathbf{x})$

What is machine learning?

```
100100011101000000101000110111010110
100100111101110000001111100110100100
100001101101111101010011100001101001
111111010000110111001010111100001011
1100111110111111100100001110110110
010000110100110110000110000100010000
01010110011001111011001110100010111
001000010101100101000001000010011110
0111010011111001011101010101011100
100010000101100010101101010111000101
01001000010010101110011100001010000
010110000010011101010010101110110001
01101111101011100010100010100010000
011010011011011010001000101111001101
000101000001100110001100100010010110
10010101010001001110010101010111101
```

ML algorithm

$f(\mathbf{x})$

Machine learning algorithms **build a mathematical model based on sample data**, known as "training data", in order to make **predictions or decisions** without being explicitly programmed to do so.

What is machine learning?

```
100100011101000000101000110111010110
100100111101110000001111100110100100
100001101101111101010011100001101001
111111010000110111001010111100001011
110011111011111100100001110110110
010000110100110110000110000100010000
0101011001100111011001110100010111
001000010101100101000001000010011110
011101001111100101110101010111100
100010000101100010101101010111000101
01001000010010101110011100001010000
010110000010011101010010101110110001
01101111101011100010100010100010000
011010011011011010001000101111001101
000101000001100110001100100010010110
10010101010001001110010101010111101
```

ML algorithm

$f(x)$

Machine learning algorithms **build a mathematical model based on sample data**, known as "training data", in order to make **predictions or decisions** without being explicitly programmed to do so.

Attributed to Arthur Samuel (1901 – 1990)
Popularized the term in 1959



Sounds like AI?

*Machine learning algorithms **build a mathematical model based on sample data**, known as "training data", in order to make **predictions or decisions** without being explicitly programmed to do so.*

Sounds like AI?

Machine learning algorithms build a mathematical model based on sample data, known as "training data", in order to make predictions or decisions without being explicitly programmed to do so.

Artificial intelligence mimics cognitive functions that we associate with human "intelligence" such as **learning**, problem solving, logic, planning, creativity, communication, consciousness, etc.

1930s

- Alan Turing: theory of computation, Turing machine, Turing test

1940s

- Artificial neuron (McCulloch & Pitts, 1943)
- Hebbian learning (Hebb, 1949)

1950s

- Dartmouth College workshop (1956): Allen Newell (CMU), Herbert Simon (CMU), John McCarthy (MIT), Marvin Minsky (MIT) and Arthur Samuel (IBM)
- Perceptron (Rosenblatt, 1957) – based on McCulloch & Pitts (1943)

1960s

- Herbert Simon: “machines will be capable, within twenty years, of doing any work a man can do”

1970s

- AI Winter: disappointments, pessimism, and funding cutbacks

1980s

- Expert systems: explicit knowledge in narrow domains
- Hopfield network (Hopfield, 1982)
- Boltzmann Machine (Ackley, Hinton, Sejnowski, 1985)
- Backpropagation: train NNs with derivatives (Linnainmaa, 1970, 1976; Werbos, 1982; Rumelhart, Hinton, Williams, 1986)
- [CNNs with representation learning \(LeCun et al., 1989\)](#)

1990s

- Slowdown in expert systems
- LSTM (Hochreiter & Schmidhuber, 1997)



Alan Turing



Frank Rosenblatt and Perceptron

Chris Bishop, 2017, Artificial Intelligence, the History and Future
https://www.youtube.com/watch?v=8FHBh_OmdsM

1930s

- Alan Turing: theory of computation, Turing machine, Turing test

1940s

- **Artificial neuron (McCulloch & Pitts, 1943)**
- **Hebbian learning (Hebb, 1949)**

1950s

- Dartmouth College workshop (1956): Allen Newell (CMU), Herbert Simon (CMU), John McCarthy (MIT), Marvin Minsky (MIT) and Arthur Samuel (IBM)
- **Perceptron (Rosenblatt, 1957) – based on McCulloch & Pitts (1943)**

1960s

- Herbert Simon: “machines will be capable, within twenty years, of doing any work a man can do”

1970s

- AI Winter: disappointments, pessimism, and funding cutbacks

1980s

- Expert systems: explicit knowledge in narrow domains
- Hopfield network (Hopfield, 1982)
- Boltzmann Machine (Ackley, Hinton, Sejnowski, 1985)
- **Backpropagation: train NNs with derivatives (Linnainmaa, 1970, 1976; Werbos, 1982; Rumelhart, Hinton, Williams, 1986)**
- **CNNs with representation learning (LeCun et al., 1989)**

1990s

- Slowdown in expert systems
- **LSTM (Hochreiter & Schmidhuber, 1997)**



Alan Turing



Frank Rosenblatt and Perceptron

Chris Bishop, 2017, Artificial Intelligence, the History and Future
https://www.youtube.com/watch?v=8FHBh_OmdsM

1930s

- Alan Turing: theory of comp

1940s

- **Artificial neuron (McCullo**
- **Hebbian learning (Hebb, 1**

1950s

- Dartmouth College workshop (MIT), Marvin Minsky (MIT) a
- **Perceptron (Rosenblatt, 19**

1960s

- Herbert Simon: "machines v

1970s

- AI Winter: disappointments,

1980s

- Expert systems: explicit kno
- Hopfield network (Hopfield,
- Boltzmann Machine (Ackley,
- **Backpropagation: train NN**
- **Rumelhart, Hinton, William**
- **CNNs with representation**

1990s

- Slowdown in expert systems
- **LSTM (Hochreiter & Schmidhuber, 1997)**

The Nobel Prize in Physics 2024



III. Niklas Elmehed © Nobel Prize Outreach

John J. Hopfield

Prize share: 1/2



III. Niklas Elmehed © Nobel Prize Outreach

Geoffrey Hinton

Prize share: 1/2

The Nobel Prize in Physics 2024 was awarded jointly to John J. Hopfield and Geoffrey E. Hinton "for foundational discoveries and inventions that enable machine learning with artificial neural networks"



Alan Turing



Frank Rosenblatt and Perceptron

ce, the History and Future

https://www.youtube.com/watch?v=8FHBh_QmdsM

"Instead of trying to produce a programme to simulate the adult mind, why not rather try to produce one which simulates the child's? If this were then subjected to an appropriate course of education one would obtain the adult brain."



Turing, A.M. (1950). Computing machinery and intelligence. *Mind*, 59, 433-460.

Definition by Tom Mitchell

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

AI and ML

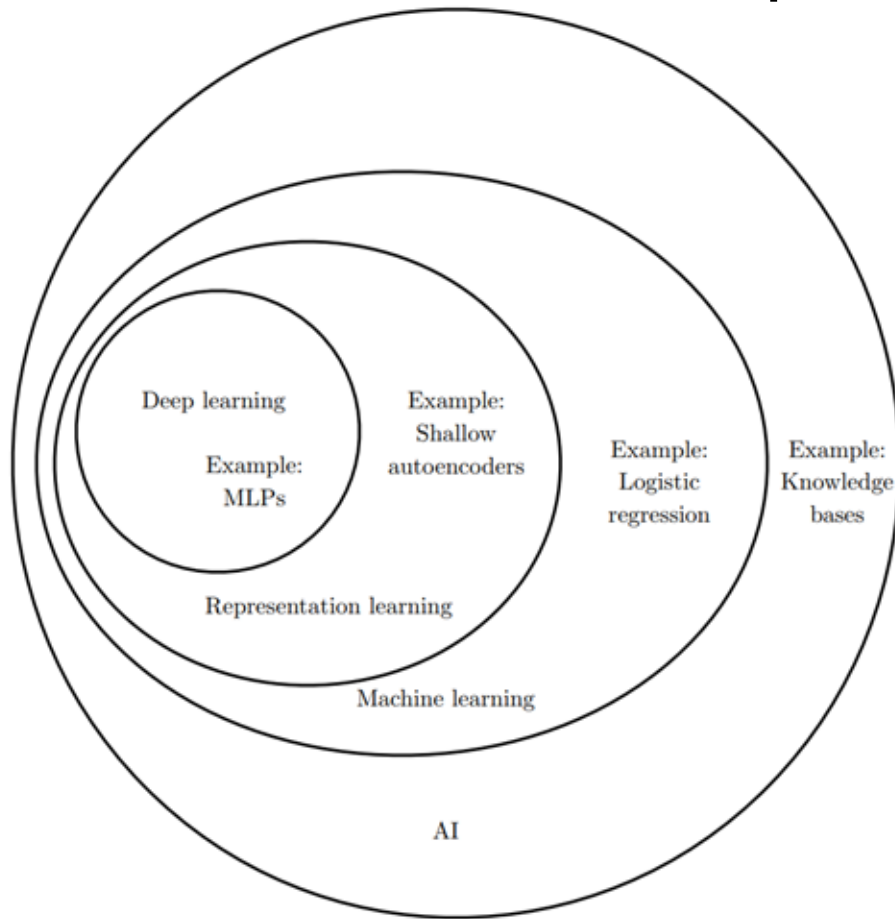
Definition by Tom Mitchell

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

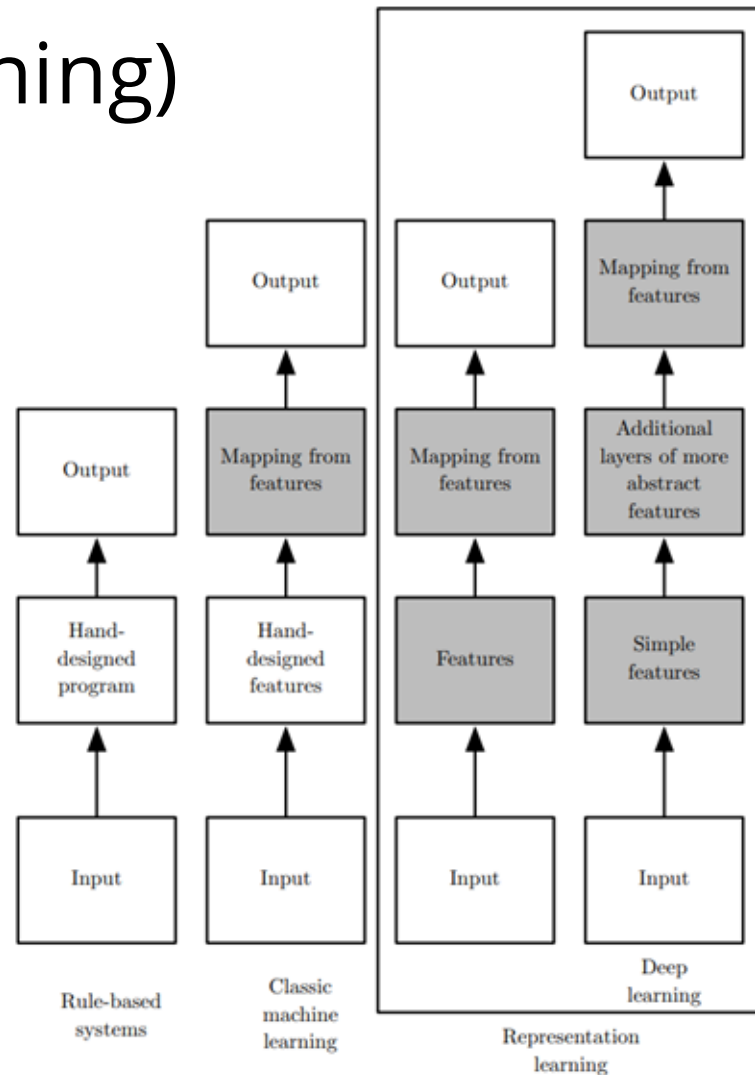
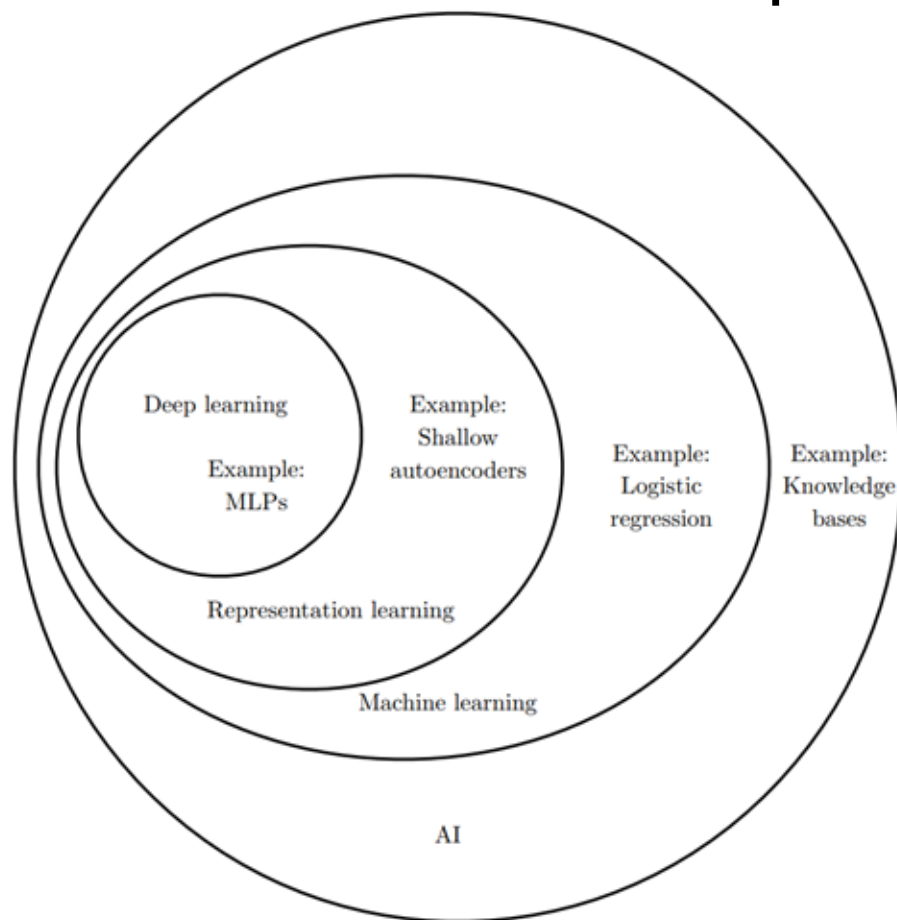
Face Detection

- ▶ E : images (with bounding boxes) around faces
- ▶ T : given an image without boxes, put boxes around faces
- ▶ P : number of faces correctly identified

AI and ML (and deep learning)

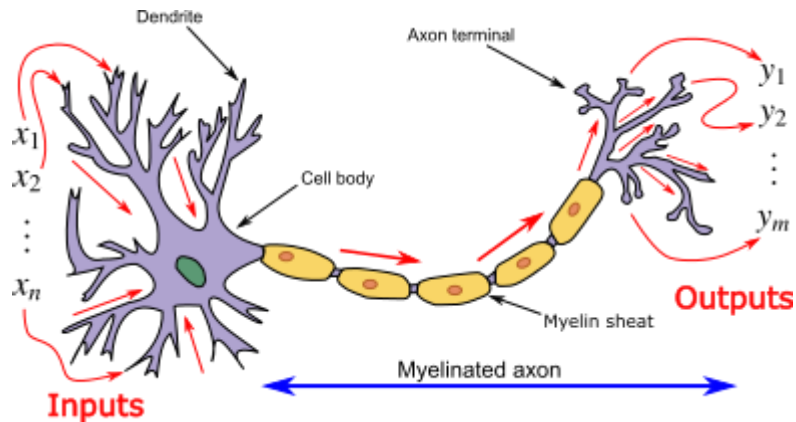


AI and ML (and deep learning)

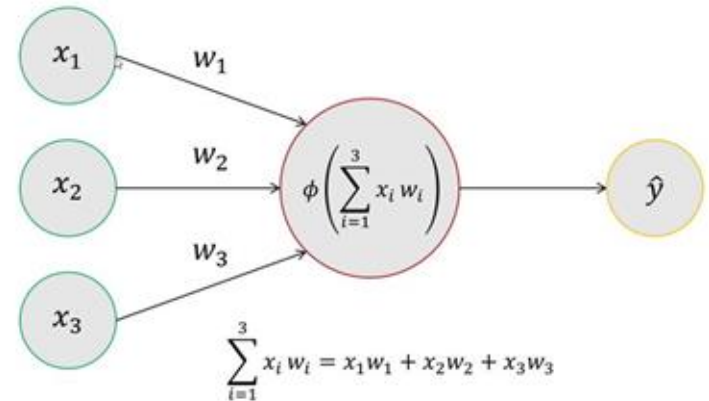


ML and neural networks

Biology

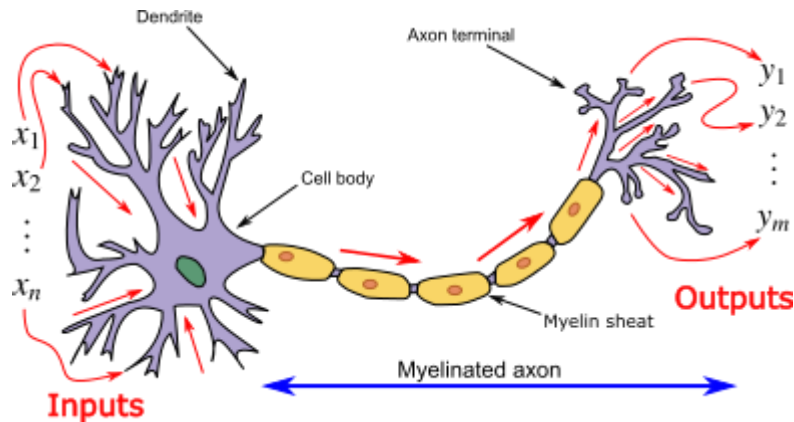


Mathematics

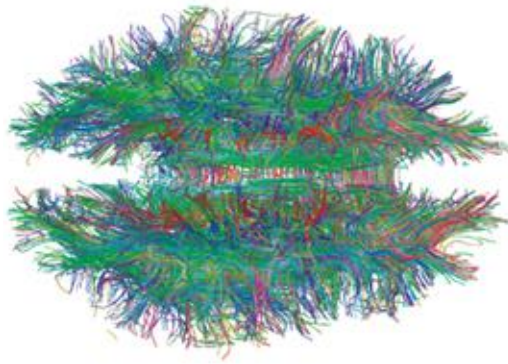


ML and neural networks

Biology

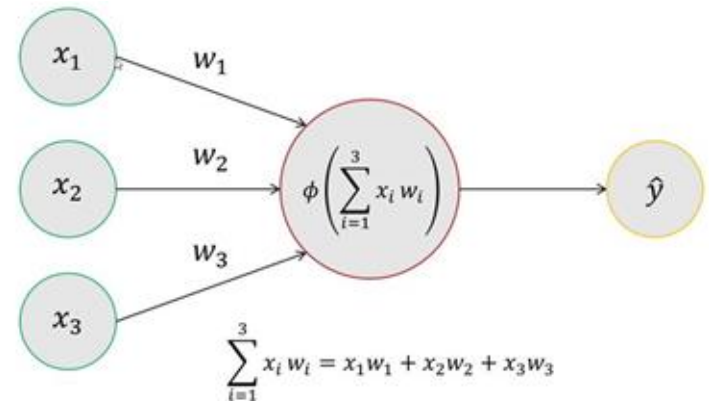


Human
brain
connectome



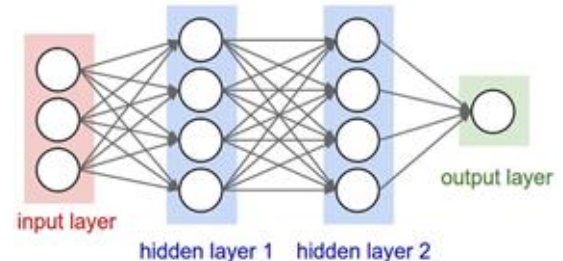
$\sim 90 \times 10^9$
neurons,
 $\sim 100 \times 10^{12}$
synapses

Mathematics



Artificial
neural
network

$\sim 175 \times 10^9$
params
(synapses)
in OpenAI
GPT-3

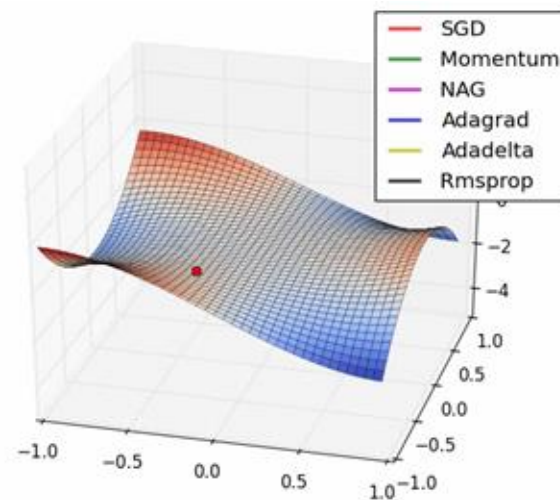
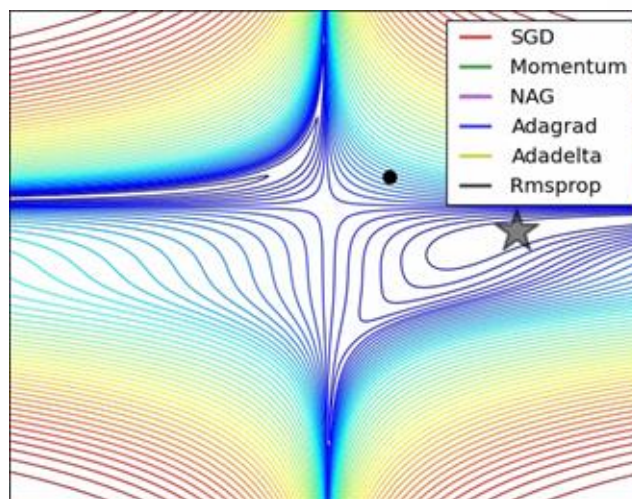


ML and optimization

In practice most of what we do is fitting differentiable functions to data using gradient-based optimization.

$$Q(\mathbf{w}) = \sum_{i=1}^N Q_i(\mathbf{w})$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \sum_{i=1}^d \nabla_{\mathbf{w}} Q_i(\mathbf{w})$$



(Ruder, 2017) <http://ruder.io/optimizing-gradient-descent/>

ML has many faces

So we can approach ML from many origins and definitions

- Computer science and AI, learning as intelligence, optimization
- Bio-inspired (artificial “neurons”), Perceptron etc.
- Psychology, neuroscience, related fields
- (Bayesian) statistics

The way people see and approach ML and the terminology they use can vary, and it's good to be aware of this.

The learning problem



Inputs

Features or measurements describing each example: sensor reading, images from experiments, etc



Outputs

The quantity that we are interested in predicting: the next timestep, how well something is controlled, etc



Hypothesis space

The family of functions that a model can represent: linear, polynomial, etc



Generalisation

How well the model performs in unseen data

Learning means searching the hypothesis space for the function that best predicts the outputs and (ideally) generalizes beyond the training examples

A big rough classification

Supervised

Labelled data → predictions

Learn from input output pairs to predict the outcomes on new inputs, we explicitly tell the model if it's doing well or not

Example: super-resolution (is the image well resolved or not)

Unsupervised

Unlabelled data → structure

Discover patterns, clusters or representations with no target labels

Example: clustering algorithms to identify related regions in the ocean

Reinforcement

Learn a decision-making policy through trial, error and reward feedback

Example: learn how to land a rocket on the surface of the moon

Supervised learning

Regression

Continuous output

Predicts a continuous output, such that the inputs can be continuously mapped

Example: Peak stress from load conditions

Classification

Discrete category

Predicts a discrete category that the data belongs to

Example: flagging a weld as defective or sound

Both learn a function of the inputs from labelled examples, but the difference is whether the output is continuous or not

Supervised learning in MAE

Some of the common applications we consider

PDE surrogates

Neural networks trained to predict field solutions, or forecast measurable quantities faster than full-order simulations

Data cleaning/ augmentation

Map raw data to calibrated physical quantities, improve resolution or compression of data

Damage classification/ Warning signals

Classify whether a structure is healthy, if a wave is dangerous, or if defects are present in a material

Generalisation vs memorisation

Training a model is good but how much should we train?

- **Underfit:** model is too simple; it misses patterns or has too much error
- **Good fit:** captures the underlying pattern and performs reasonably well on unseen data
- **Overfitting:** it captures the training data too well. It specializes too much and performs poorly on data beyond training

Training set

Test

The model learns in the training set and we hold out a test set to give an honest estimation of the performance. Typically training set ~80% of data

Unsupervised learning

Learning without labels! The goal is to find structures, patterns or compact representations of the data itself.

Why is this important?

Feature discovery: finding nonlinear relationships between different data groups, e.g. grouping ocean regions based on a nonlinear combination of their properties

Anomaly detection: highlighting unusual behaviour/patterns, data that does not quite fit an underlying distribution

Dimensionality reduction: finding compact, informative representations of the data when the system is too complex, e.g. autoencoder to find a latent space representation

Unsupervised learning in MAE

Some of the common applications we consider

Clustering flow regimes

Grouping the behaviour in turbulent flows without a prior knowledge of what links these regions

Autoencoders and PCA

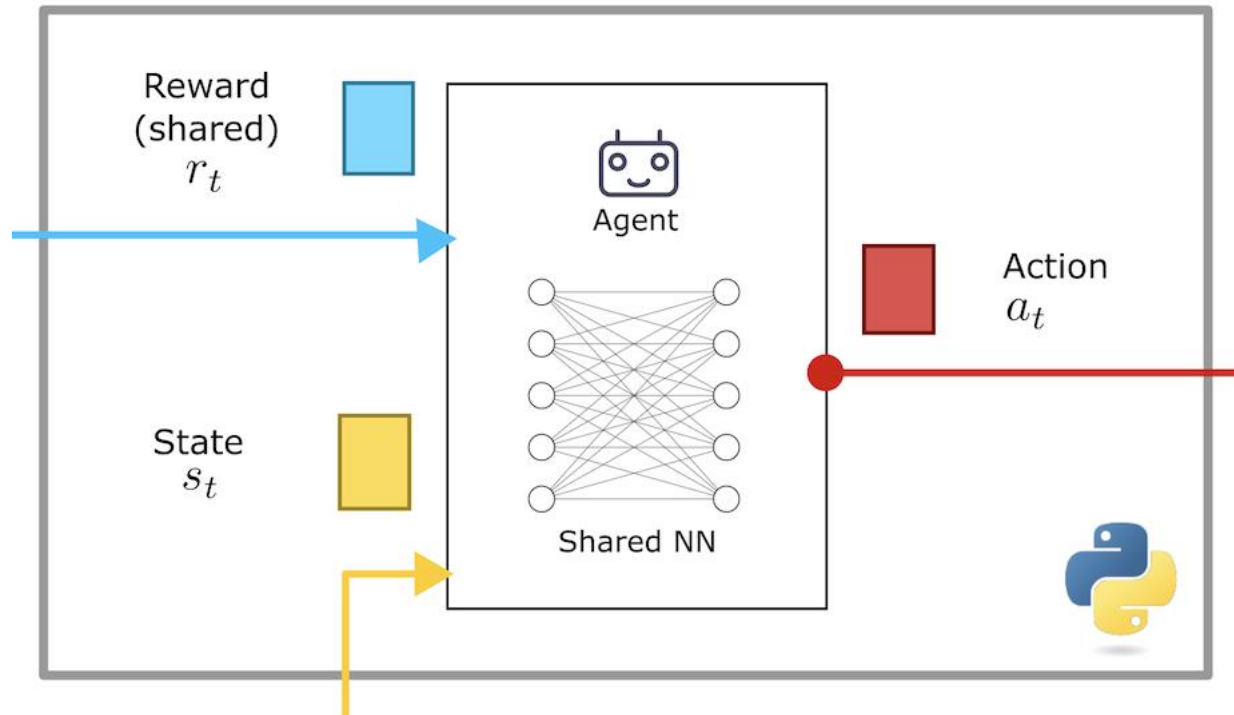
Find a latent (low-order) representation of the data to identify what structures contain most of the variance in the data, e.g. minimal models of bending beams

Anomaly detection

Identify sensor readings that deviate from the normal learned behaviour ahead of failure

Reinforcement learning

Learning through interaction



The agent takes actions in an environment; the environment evolves and returns a new state and a reward signal. Over repeated interaction, the agent learns a policy that maximises cumulative reward.

Reinforcement learning in MAE

Some of the common applications we consider

Turbulence control

Reducing the drag on airplane wings or increasing mixing in a channel with filaments

Robotic manipulation

Learning task for grasping, assembling and manipulation tasks

Process control

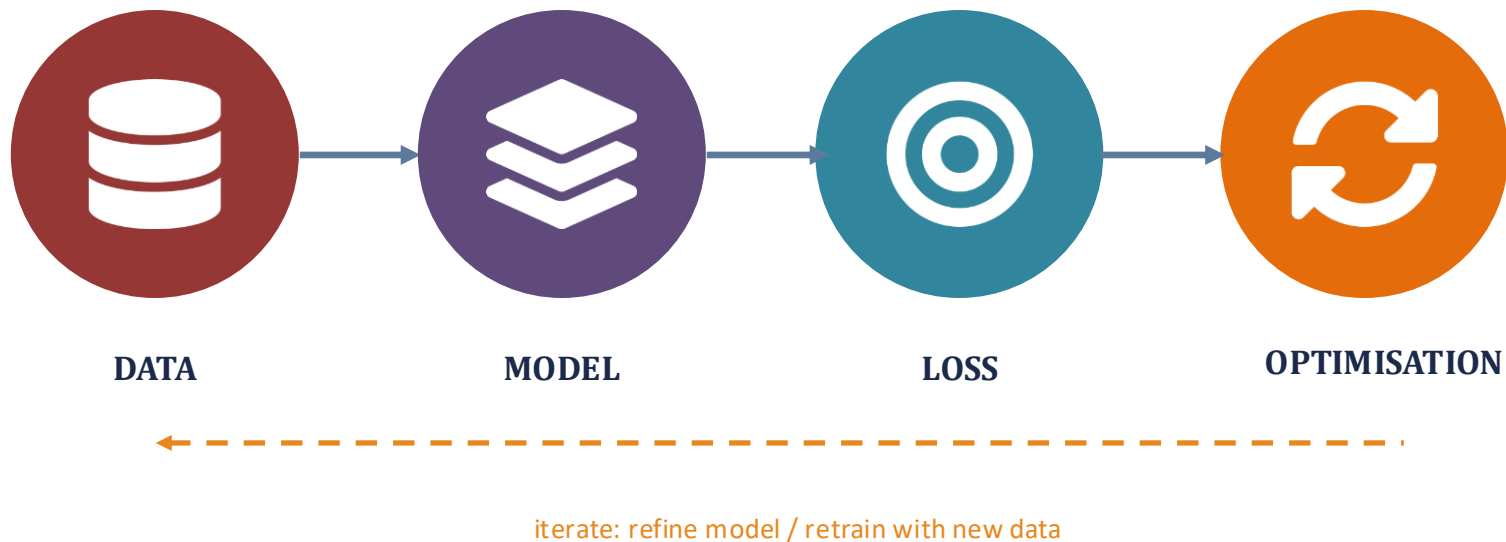
Adaptive control of chemical, energy, or manufacturing processes under changing conditions.

Which strategy fits your problem?

Question	Supervised	Unsupervised	RL
Do you have labelled data?	Yes	No	No. Reward signal instead
Is the goal prediction or discovery?	Prediction	Discovery	Decision-making
Is there a sequential decision process?	No	No	Yes

How do we actually build these?

Typical ML pipeline: data, model, loss, optimisation



We need enough data, a good enough model, a loss (objective) to minimise/maximise and an optimiser to find the next step.

Stage 1: Data collection

Important to have sufficient data of good enough quality



- Data is typically expensive (or limited) we want to use mostly to train.
- We want to keep a part of the data just for validation purposes
- We often want a held-out set for pure testing evaluation, data not seen on training nor evaluation

Careful with:

- Noisy data: even small amounts of noise can compromise the whole pipeline
- Errors in the input data and data-leakage, e.g. future data being use in present predictions
- Non stationary conditions, e.g. trasients or systems that drift from the original state

Stage 2: Choosing the model

We have loads of choices

Linear models

e.g. PCA, linear regression. These are data-driven but not ML. They are simple, interpretable and work well as baselines to beat

Neural networks

e.g. CNN, GNN, U-Nets. These are part of “deep learning”, they have hidden layers and are universal approximators for complex high-dimensional mappings. Typically, not interpretable “black box”

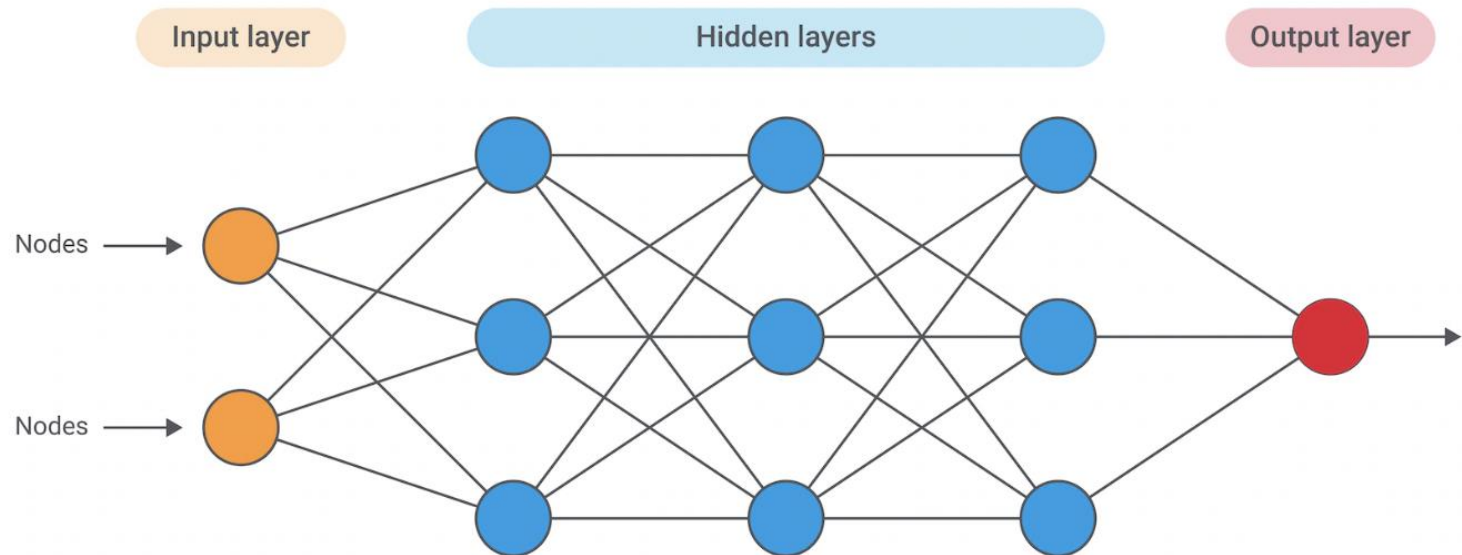
Symbolic regression

e.g. SINDY, PySR. Search for closed form expressions to the data that we have. They are interpretable, but might struggle with very high dimension, or data which can be described by many functions

Typically we trade interpretability with expressive power

Stage 2: Choosing the model

Neural networks are the cornerstone of ML



Neuron

Computes a weighted sum of its inputs, adds a bias, then passes the result to an activation function

Weights and biases

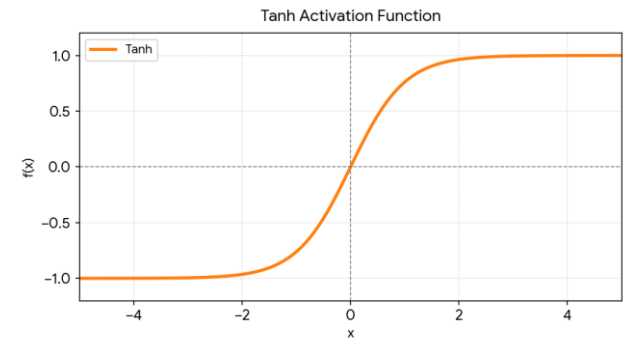
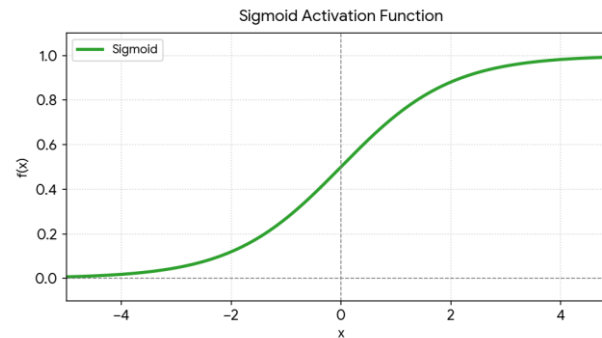
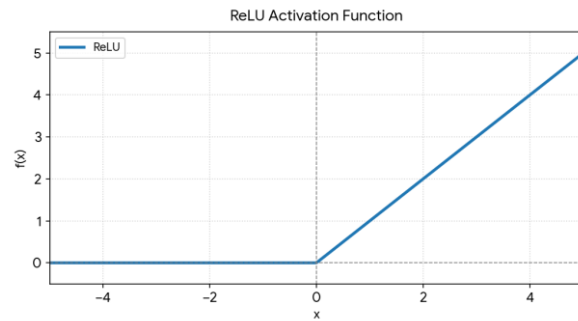
The learnable parameters of the network. Adjusted through the training

Depth and Width

Stacking layers (depth) and adding neurons per layer (width) to increase representation capabilities

Stage 2: Choosing the model

Activation functions are what introduces the nonlinearity. **Without nonlinear activation, stacking any number of layers collapses to a single linear model.**



ReLU

Default activation in
feedforward networks

Sigmoid

Almost always the output
layer in a binary
classification problem. It
provides a probabilistic
interpretation between 0
and 1

tanh

Typically used in
recurrent neural
networks. Zero-
centered alternative to
the Sigmoid

Stage 3: Loss function

Quantifying the error. i.e. what is it that we want our model to achieve

MSE

Mean-squared error

One of the most popular measures in **regression**, penalises the distance to the target assuming Euclidian spaces.

Comments: does not promote sparsity but is sensitive to large errors

Cross-Entropy

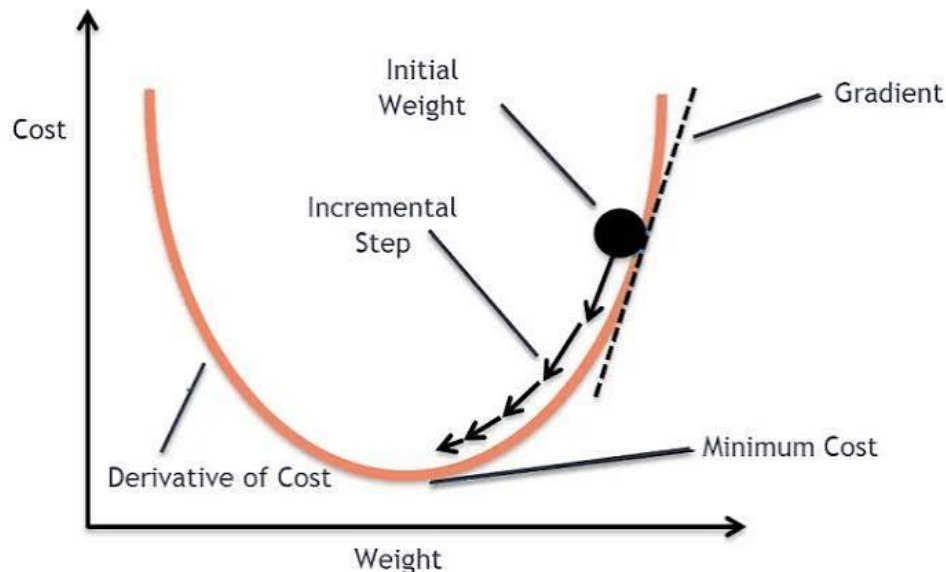
One of the most popular measures in **classification problems**. It penalises confident but wrong class probabilities while it rewards well-calibrated predictions

Designing the reward can be critical for our goals, e.g. if we have a material, and we want to ensure defects are accurately captures, we might choose a loss that particularly penalises this. If we add the terms of a governing equation in the loss, this is PINNs

Stage 4: Optimisation

After a pass through the model (typically neural network), we need to backpropagate the loss to update the weights.

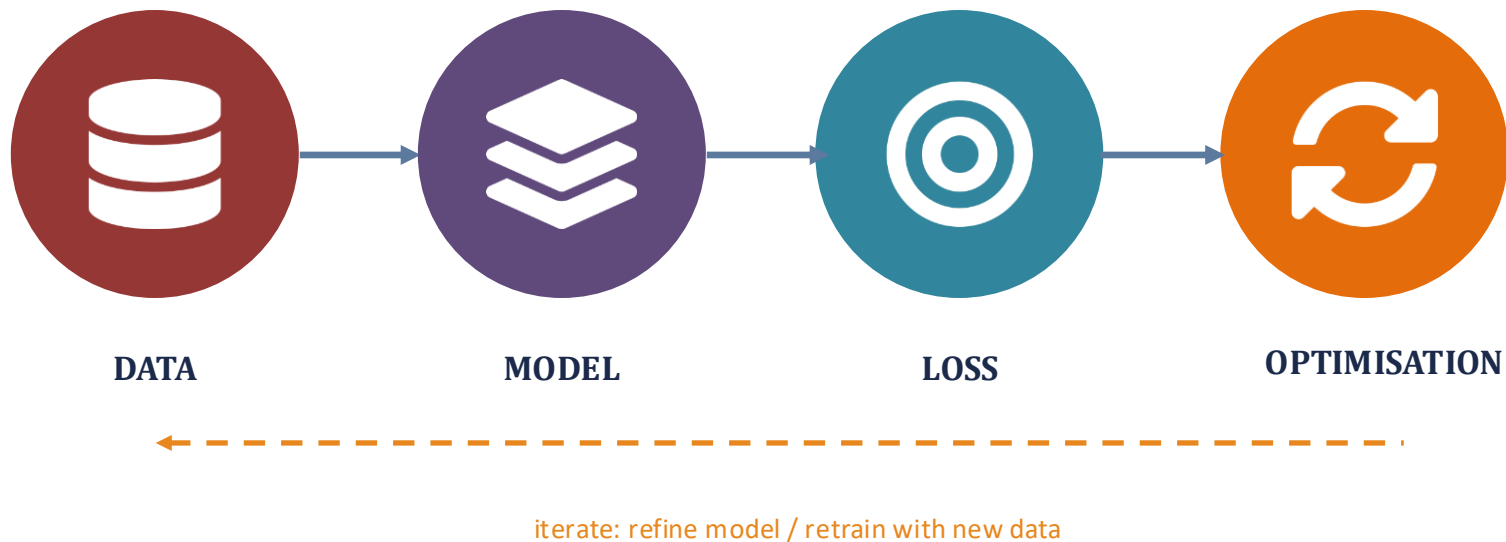
In practice:



- **Iterative refinement** modify the parameters iteratively to reduce the loss
- **Learning rate** how much we update the weights. Very small steps are stable but slow and can stall, very large steps can become unstable and overshoot
- **Over/underfitting** we typically choose the best model obtained during training. Stop training early if needed to avoid overfitting.

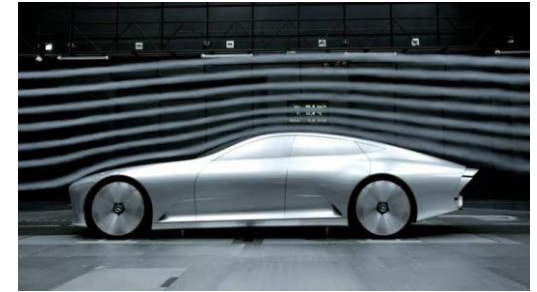
Stage 5: fine-tuning

Typically, there are many (hyper) parameters involved in the training e.g. learning rates, layer sizes and shapes, etc.



If cost allows, we can fine-tune the setup to obtain even better performing models.

A pipeline example



We want to predict the air resistance (drag) in a car

1. Data collection and curating

Collection of simulation/experimental data, create a database, data-loader, clean noisy data for many geometries and conditions

2. Model selection

A simple multilayer perceptron (MLP) where all neurons are connected for regression

3. Loss

MSE on the drag reduction from the network vs. measured one (supervised learning task)

4. Optimisation

Adam (commonly used optimiser), trained for a fixed number of epochs (passes through the data)

Common pitfalls

Things to keep in mind (not an exhaustive list):

Keep track of the data

Ensure a proper split of the data and that the training dataset is big enough. Make sure validation data is not leaked into training and that the inputs/outputs of the network are what you want it to be

Example: performance degrades in test data because the data distribution is different than the training

Careful with the data split (and using the right model)

MLP does not have any geometrical information, CNNs need uniformly distributed data for convolutions, keep the physical data with the right shape

Example: A CNN can't perform well on unstructured or sparse data

ML statistically works

The optimization process has often many local optima, we might find several models which perform similarly well, some that perform very well and some that don't. Important to check training with different seeds and take the best models. Training just one model might not guarantee beating a baseline

Example: training 20 autoencoders on the same data and selecting the best performing one

Evaluating the models

We work in physical and engineering setups. Have low error metrics is a first step but we need to consider physical consistency

Is the model satisfying physical constraints?

Do the models capture the right amount of energy in the system?
How well do they satisfy the governing equations?

Does it generalise out of distribution?

We train for a particular set of physical parameters, but does our model work beyond these? Why? Why not?

Choose a strong baseline

Training ML models can be expensive, and using modern mathematical tools can be as optimal as ML. Make sure you always have a baseline and assess the cost of your training

nature machine intelligence

Analysis

<https://doi.org/10.1038/s42256-024-00897-5>

Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations

This course

Two weeks, project based to give a hands-on start into something you are interested and hopefully can carry to your own research.

- **This year:** three lecture topics (6h per topic)
 - Time series and symbolic regression
 - Reinforcement learning
 - Introduction to uncertainty quantification
- Lectures in the morning 9-12 (see lecture rooms in the calendar invites)
- Individual project work supported by amazing TAs in the afternoon, but self-learning is encouraged. Dedicated time to learn about ML and we are happy to help!
- Presentations on Friday 28th of August with the progress. Gift card for the best project!

Why these topics?

General enough to cover a wide range of topics in Mech and Aero, e.g. solid mechanics, fluid mechanics and robotics

- **Time series and symbolic regression:** modelling dynamics over time and discovering interpretable, closed-form relationships in the data
- **Uncertainty quantification:** knowing how much a prediction can be trusted, essential for safety design and decision making
- **Reinforcement learning:** learning to interact with systems to achieve goals, closing the loop from model to decision.

Some logistics



Logistics Recap

- MaSC portal, if you don't have access to it, let us know. Google Collab or local installations are also possible but less supported
- GitHub: ideally create a Github repository with your code. Good practice for version control
- Don't wait until a particular lecture to get started. Selected a project from the shared list and let us know by the end of tomorrows lecture!



Group Project

- Scope: apply course methods to an engineering problem. Can be your own research or a stepping stone.
- Objectives: demonstrate a working data-driven pipeline to bring an ML project to reality
- Deliverables: code for personal use and final public presentation
- Format: 15-min presentation to the full cohort on the final day

Today's logistics



- Project list distributed via email alongside sign-up sheet. If you haven't chosen a project please do so.
- If you know who you want to work with, please add names of the group members to the project.
- If you do not have a preference, sign-up in the "open for projects column". You can specify a preference and we will set up the groups.

Project list Summer School in Hands-on Scientific ML 2026

Project 1. Airfoil Shape Optimization using Reinforcement Learning

Airfoil shape optimization is a classical aerospace engineering problem. The objective is to modify the geometry of an airfoil to improve aerodynamic performance, such as maximizing the lift coefficient, minimizing the drag coefficient, maximizing lift-to-drag ratio, etc. Traditionally this problem is solved using gradient-based optimization or gradient-free algorithms. Recently, reinforcement learning (RL) has emerged as a promising alternative because it can learn sequential design strategies instead of treating every optimization independently. Instead of searching the design space from scratch for every new case, an RL agent learns how to improve shapes through experience, making it suitable for exploring promising designs. Suggested code: XFOIL <https://web.mit.edu/drela/Public/web/xfoil/>

Project 2. Symbolic regression for nonlinear benchmarks

Machine learning models are typically black boxes, whereas in science and engineering the governing equations themselves are often of interest. Symbolic regression searches for interpretable mathematical expressions that describe data. The SINDy framework (Sparse Identification of Nonlinear Dynamics) [1] recovers governing equations by sparse regression on a library of candidate terms, while genetic-programming approaches such as PySR [2] search expression space directly.

Most demonstrations of these methods use clean data generated from textbook equations, where the correct answer is known in advance. This project instead uses established public benchmark datasets from the nonlinear system identification community [3]. These are measurements from real

What to do this afternoon?



- Start planning the required steps for the selected problem. Make a roadmap of the steps you'd need to take, and the ML approaches you'll require, e.g. supervised learning, define the inputs and outputs of the model, etc
- Prepare the physical aspects of the project, e.g. set up the physical problem and start with the data generation
- If you are doing a **reinforcement learning** problem, prepare the environment that is going to interact with the agent
- If you have time, prepare the slides about the physical background for the presentation

Thank you for your attention!

Happy to take any questions